

Pill Adherence App Complete Build Guide

Pill Adherence App — Complete Build Guide

Handoff guide for building the Flutter app (excludes camera/pill classification — that's being built separately)

This guide assumes zero prior context. Follow it top to bottom.

Part 1: Environment Setup

1.1 Install Flutter SDK

1. Download from <https://docs.flutter.dev/get-started/install/windows> (or macOS/Linux page if applicable)
2. Extract to a simple path with no spaces — e.g. `C:\src\flutter`
3. Add `C:\src\flutter\bin` to your system PATH:
 - Windows key → search “Environment Variables” → Edit the system environment variables
 - Environment Variables → under “User variables”, select **Path** → Edit → New → paste
`C:\src\flutter\bin`
 - OK on all dialogs
4. **Restart your terminal/VS Code completely** (PATH changes don't apply to already-open terminals)
5. Verify: `flutter --version` should print a version number, not an error

1.2 Install Android Studio (for the Android SDK)

1. Download from <https://developer.android.com/studio>
2. Run the installer, choose **Standard** setup
3. Let it finish downloading the SDK/platform tools (10–15 min)
4. Open Android Studio once → **More Actions** → **SDK Manager** → **SDK Tools tab**
5. Check **Android SDK Command-line Tools (latest)** → Apply

1.3 Verify everything

Run:

```
flutter doctor
```

You want green checkmarks on: - Flutter - Android toolchain - Connected device (plug in an Android phone with USB debugging enabled, or use Android Studio's built-in emulator)

Ignore any “Visual Studio not installed” warning — that's only for Windows desktop apps, not relevant here.

1.4 Install VS Code extensions

In VS Code, install the **Flutter** extension (this auto-installs the **Dart** extension too) from the Extensions marketplace.

Part 2: Create the Project

```
flutter create pill_app
cd pill_app
flutter pub add hive hive_flutter flutter_local_notifications provider path_provider intl
```

Open the `pill_app` folder in VS Code.

Sanity check — run this and confirm the default counter app launches on your device:

```
flutter run
```

If this works, your environment is fully ready. If not, fix it before continuing — don't build on top of a broken setup.

Part 3: What You're Building (Scope)

You are building **everything except the camera/pill-image classification**, which a teammate is handling separately. Your job:

1. **Data model + local storage** — save medicine info on the device (no backend, no login, no internet needed)
2. **Add medicine + schedule screen** — where a name (eventually passed in from the camera teammate's classifier) and dose times get entered and saved
3. **Local notifications** — reminders that fire at the scheduled times, even with the app closed
4. **"Today's medications" checklist** — a home screen showing what's due and letting the user mark it as taken

Integration point with your teammate: their code will eventually hand you a `String` `medicineName` and `String imagePath` (the photo they took). For now, build and test everything using a **hardcoded/typed-in name** instead — don't wait on their code to be ready. Wiring their real output in is a 10-minute change once both sides are done.

Part 4: Build It — Step by Step

Step 1: Create the data model

Create a new file `lib/models/medicine.dart`:

```
import 'package:hive/hive.dart';

part 'medicine.g.dart';

@HiveType(typeId: 0)
class Medicine extends HiveObject {
  @HiveField(0)
  String name;

  @HiveField(1)
  String imagePath;

  @HiveField(2)
  List<String> doseTimes; // e.g. ["08:00", "20:00"]
}
```

```

    @HiveField(3)
    DateTime? lastTaken;

    Medicine({
      required this.name,
      required this.imagePath,
      required this.doseTimes,
      this.lastTaken,
    });
  }
}

```

Generate the required adapter file:

```
dart run build_runner build
```

This creates `medicine.g.dart` automatically — don't write this file by hand.

Step 2: Initialize Hive in main.dart

Replace the top of `lib/main.dart`:

```

import 'package:flutter/material.dart';
import 'package:hive_flutter/hive_flutter.dart';
import 'models/medicine.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  Hive.registerAdapter(MedicineAdapter());
  await Hive.openBox<Medicine>('medicines');
  runApp(const MyApp());
}

```

Step 3: Build the “Add Medicine” screen

Create `lib/screens/add_medicine_screen.dart`. This screen needs: - A text field for medicine name (later this gets pre-filled by the camera teammate's output — for now, just let the user type it) - A way to add one or more dose times (use Flutter's built-in `showTimePicker`) - A save button that creates a `Medicine` object and saves it to the Hive box:

```

final box = Hive.box<Medicine>('medicines');
final medicine = Medicine(
  name: nameController.text,
  imagePath: '', // will come from camera teammate later
  doseTimes: selectedTimes, // list of "HH:mm" strings you collected
);
await box.add(medicine);

```

Test this fully before moving on — add a medicine, restart the app, confirm it's still there (this proves Hive is persisting data correctly).

Step 4: Add local notifications

Create `lib/services/notification_service.dart`:

```

import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:timezone/timezone.dart' as tz;
import 'package:timezone/data/latest.dart' as tzdata;

class NotificationService {
  static final _plugin = FlutterLocalNotificationsPlugin();
}

```

```

static Future<void> init() async {
  tzdata.initializeTimeZones();
  const androidSettings = AndroidInitializationSettings('@mipmap/ic_launcher');
  await _plugin.initialize(const InitializationSettings(android: androidSettings));
}

static Future<void> scheduleDaily(int id, String title, String body, int hour, int minute) async {
  await _plugin.zonedSchedule(
    id, title, body,
    _nextInstanceOfTime(hour, minute),
    const NotificationDetails(
      android: AndroidNotificationDetails('med_channel', 'Medication Reminders'),
    ),
    androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,
    matchDateTimeComponents: DateTimeComponents.time,
  );
}

static tz.TZDateTime _nextInstanceOfTime(int hour, int minute) {
  final now = tz.TZDateTime.now(tz.local);
  var scheduled = tz.TZDateTime(tz.local, now.year, now.month, now.day, hour, minute);
  if (scheduled.isBefore(now)) {
    scheduled = scheduled.add(const Duration(days: 1));
  }
  return scheduled;
}
}

```

Add the `timezone` package:

```
flutter pub add timezone
```

Call `NotificationService.init()` in `main()` before `runApp()`. Then, in your Add Medicine save logic, call `scheduleDaily()` once for each dose time selected.

Critical test: schedule a notification for 1-2 minutes in the future, then **close the app completely and lock your phone**. Confirm the notification still fires. This is the single most common thing that breaks silently in hackathon demos — don't skip this test.

Step 5: Build the “Today’s Medications” checklist screen

Create `lib/screens/home_screen.dart`:

```

ValueListenableBuilder(
  valueListenable: Hive.box<Medicine>('medicines').listenable(),
  builder: (context, Box<Medicine> box, _) {
    final meds = box.values.toList();
    return ListView.builder(
      itemCount: meds.length,
      itemBuilder: (context, i) => CheckboxListTile(
        title: Text(meds[i].name),
        subtitle: Text(meds[i].doseTimes.join(', ')),
        value: meds[i].lastTaken != null &&
          _isToday(meds[i].lastTaken!),
        onChanged: (_) {
          meds[i].lastTaken = DateTime.now();
          meds[i].save();
        },
      ),
    );
  },
)

bool _isToday(DateTime date) {

```

```
final now = DateTime.now();  
return date.year == now.year && date.month == now.month && date.day == now.day;  
}
```

`ValueListenableBuilder` automatically rebuilds this screen whenever Hive data changes — no manual refresh logic needed.

Step 6: Wire up navigation

In `main.dart`, set `HomeScreen` as your initial route, with a floating action button or app bar button that navigates to `AddMedicineScreen`.

Part 5: Integration With the Camera Teammate

Once their classifier is working, the only change needed: instead of the user typing the medicine name in Step 3's text field, pre-fill it with the `String medicineName` their code produces (and pass along their `String imagePath` too). Everything else — storage, scheduling, notifications, checklist — stays exactly the same. This is why it's safe to build your half independently and integrate at the end.

Agree on the exact function signature with them ahead of time, e.g.:

```
void onMedicineIdentified(String medicineName, String imagePath) {  
  // pre-fill the Add Medicine screen with these values  
}
```

Part 6: Testing Checklist Before Submission

Run through this full flow on a real device, fresh install: - [] Open app, add a medicine with a name and 1-2 dose times - [] Confirm it appears in the "today's medications" list - [] Close and reopen the app — data persists - [] Schedule a notification 1-2 min out, lock the phone, confirm it fires - [] Mark a medicine as "taken" — checkbox updates and persists after restart - [] (Once integrated) confirm the camera teammate's output correctly pre-fills the Add Medicine screen

Part 7: If You Get Stuck

- **Hive adapter errors:** re-run `dart run build_runner build --delete-conflicting-outputs`
- **Notifications not firing on Android 13+:** you may need to request the `POST_NOTIFICATIONS` permission at runtime — check the `flutter_local_notifications` package's README for the current permission request snippet, since Android's notification permission requirements have changed across versions
- **App won't run on device:** re-check `flutter doctor`, make sure your device shows under "Connected device"